Statistical Independence Aware Caching for LLM Workflows

Yihan Dai
yhdai25@stu.pku.edu.cn
Key Lab of HCST (PKU), MoE, SCS, Peking University
Beijing, China

Dimitrios Stamatios Bouras
2501112125@stu.pku.edu.cn
Key Lab of HCST (PKU), MoE, SCS, Peking University
Beijing, China

Haoxiang Jia
haoxiangjia@stu.pku.edu.cn
Key Lab of HCST (PKU), MoE, SCS, Peking University
Beijing, China

Sergey Mechtaev
mechtaev@pku.edu.cn
Key Lab of HCST (PKU), MoE, SCS, Peking University
Beijing, China

Abstract

Large language models (LLMs) inference is both expensive and slow. Local caching of responses offers a practical solution to reduce the cost and latency of LLM queries. In research contexts, caching also enhances reproducibility and provides flexibility for experimentation. However, naive reuse of cached responses compromises statistical independence, a critical property for probabilistic workflows. In applications of LLM for code, it underpins performance metrics such as Pass@k and uncertainty estimation, as well as algorithms like program repair loops and retries. Existing LLM caching systems lack ways to enforce statistical independence constraints. To address this, we introduce Mnimi, a cache design pattern that supports modular LLM workflows while ensuring statistical integrity at the component level. Its core innovation lies in encapsulating statistical constraints within the type of LLM references, allowing users to manage and transform these types according to the scope and requirements of their algorithm. We implemented this design pattern in Python using a combination of decorators and iterators over infinite sequences. A case study on SpecFix, a recent automated program specification repair system, highlights how Mnimi improves reproducibility, ease of debugging, time and cost efficiency while preserving statistical correctness.

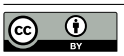
CCS Concepts

• **Computing methodologies** → **Natural language processing**; • **Software and its engineering** → **Abstraction, modeling and modularity**; **Designing software**.

Keywords

Large language models, cache, statistical independence

ACM Reference Format:
Yihan Dai, Dimitrios Stamatios Bouras, Haoxiang Jia, and Sergey Mechtaev. 2026. Statistical Independence Aware Caching for LLM Workflows. In *3rd International Workshop on Large Language Models For Code (LLM4Code '26), April 12–18, 2026, Rio de Janeiro, Brazil*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3786181.3788729



LLM4Code '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2412-1/2026/04
https://doi.org/10.1145/3786181.3788729

1 Introduction

The use of LLMs in software engineering is rapidly expanding; however, this comes with substantial costs in time, money, and energy due to LLM inference. LLM backend optimizations [14] alleviate these issues but are model-specific, requiring extensive engineering or proprietary hardware access. A practical alternative is client-side caching of LLM responses [2, 10, 18], which avoids redundant queries, improving speed, cost-efficiency, and enabling deterministic debugging, without altering the LLM. In research contexts, it enhances reproducibility and facilitates experimentation.

A typical client-side caching approach, as implemented in the popular LangChain framework [5], associates each prompt with a single LLM response. Whenever the same prompt is encountered again, the cached response is returned instead of generating a new one. This design conflicts with statistical independence required by many algorithms. In LLMs for code, widely adopted metrics like Pass@k [6] rely on generating statistically independent samples to estimate the probability of a solution's success. Similarly, methods for calibrating model uncertainty [11, 15, 17], as well as iterative repair and refinement pipelines [12, 16, 20], inherently depend on the model's ability to produce varied outputs for the same prompt. Meanwhile, a naive cache collapses this necessary variation.

A straightforward approach to address this problem is to cache a sequence of independent responses for each prompt and replay them during every application run, instead of caching a single response. While this ensures statistical independence within each run, it has practical drawbacks. First, it is inefficient because it does not reuse samples within the run. Second, it undermines reproducibility and debugging, as even small changes to the program can disrupt the sequence of responses being replayed. Cache Saver [18] alleviated this issue by introducing explicit namespaces, where each namespace stores independent responses for a given prompt. However, this approach prohibits reuse within a namespace by design, limiting flexibility for users in controlling granularity of sample reuse. Furthermore, LLM workflows are often modular and deeply nested and therefore sampling constraints are also modular and nested. The requirement to explicitly maintain statistical independence namespaces harms modularity.

LN	Statement	Iterator State Transition	Cache Status	LLM
1	`prompt = "Choose a number between 1 and 1000."`			
2	`model = ConcreteLLMProvider("model_name", 1.0)`			
3	`n1 = next(model.sample(prompt))` `#"92"`	`[→"92", ...]` `["92", →...]`		`"92"`
4	`n2 = next(model.sample(prompt))` `#"747"`	`[→"747", ...]` `["747", →...]`		`"747"`
5	`model = Repeatable(model)`			
6	`n3 = next(model.sample(prompt))` `#"131"`	`[→"131", ...]` `["131", →...]` (create new iterators from cache)	`miss one {"1":"131"}`	`"131"`
7	`n4_list = list(islice(model.sample(prompt), 2))` `#["131", "561"]`	`[→"131", "561", ...]` `["131", "561", →...]` (create new iterators from cache)	`hit one, miss one {"1":"131","2":"561"}`	`"561"`
8	`model = Independent(model)`			
9	`n5_list = list(islice(model.sample(prompt), 2))` `#["131", "561"]`	`[→"131", "561", ...]` `["131", "561", →...]` (share the same iterator)	`hit two {"1":"131","2":"561"}`	
10	`n6_list = list(islice(model.sample(prompt), 2))` `#["452", "980"]`	`["131", "561", →"452", "980", ...]` `["131", "561", "452", "980", →...]` (share the same iterator)	`miss two {"1":"131","2":"561","3":"452","4":"980"}`	`"452","980"`

Figure 1: An example showing the different behaviors of `Independent` vs. `Repeatable`. In the "Iterator State Transition" column, the first sub-row of it shows the sequence state returned by calling `sample` and the second sub-row shows the state after iterating to retrieve elements, while "→" denotes the pointer to the next element inside the iterator.

To address these limitations, we introduce Mnimi[1], a new caching design pattern that supports modular LLM workflows, while ensuring statistical integrity at the component level. At its core, the idea is to encapsulate statistical independence constraints as types of LLM references: `Repeatable`, ensuring repeated responses for each LLM query, and `Independent`, ensuring independent responses for each query. This design pattern promotes modularity by enabling transformations of LLM reference types through decorators [8], effectively allowing the user to specify statistical constraints as easily as "wrapping a gift". These layers form a coherent hierarchy governing stability within a program scope, diversity across scopes, and continuity across runs — capturing the complete statistical semantics that a typical application requires.

To show Mnimi's applicability in the context of LLM for code, we conducted a case study on SpecFix [12], a recent tool for repairing ambiguous programming problem descriptions. Its heavy reliance on stochastic LLM calls makes runs expensive and hard-to-replicate, with variations in generated tests, programs, and repaired description. A naive LLM caching cannot be adopted, because of SpecFix's reliance on semantic entropy [13] and Pass@k computation, as well as its main repair loop, all requiring independent samples. To resolve this tension, we replaced the original LLM calls in SpecFix with Mnimi, while carefully encoding the statistical independence constraints via the type of references to the LLM object. Experiments show that Mnimi improves execution time and inference cost over the original tool, and enables bit-reproducibility, while requiring only minor changes to SpecFix source code.

A Python implementation of Mnimi, that supports multiple providers, cache slicing (extracting a subset of cache), and batch sampling, as well as our SpecFix integration, is available at

https://github.com/msv-lab/mnimi

[1]Mnimi derives from the Greek word *μνήμη* (*mnēmē*), meaning "memory." In Greek mythology, Mnēmē is one of the three primordial Muses, symbolizing remembrance, the foundation of knowledge and creativity. This aligns our goal to bring structured memory to LLM workflows, balancing reuse and independence.

2 Mnimi

The challenge with statistical independence arises when an application repeatedly queries an LLM using the same prompt, i.e., samples multiple values from the same distribution. The following Python snippet illustrates an LLM-based guessing game

```
template = "Choose an integer from {interval} for {user}
    to guess."
user = ['Alice', 'Bob', 'Alice']
interval = "[1, 1000]"
responses = []
for user in users:
  prompt = template.format(user=user, interval=interval)
  response = model.sample(prompt) # queries an LLM
  responses.append(response)
```

In this example, a shared prompt template is used across different games to generate guessing numbers within the given interval. A key characteristic of the input `users` is that it may include duplicate usernames (e.g., `'Alice'`), reflecting the scenario where a single user engages with multiple distinct games.

The critical issue lies in whether the responses for duplicated usernames (e.g., Alice's first and third entries) are repeated or independent, as this directly impacts Alice's strategy for guessing the number. Assume that a fair game requires that responses are independent, that is, every time an integer is requested from the LLM for a specific user, it will ensure that the response for this time is independent of all previous responses[2]. However, if the application is changed to use a naive cache that simply returns a cached response to the same prompt, the responses will then be repeated. Since statistical independence is a prerequisite of the program above, using naive cache contradicts the intended semantics. Thus, the use of a cache in this scenario requires encoding and respecting algorithm's sample independence constraints.

Let's extend the example so that each user plays several rounds of the game, possibly guessing numbers from different intervals:

[2]note that independence does not imply uniqueness because of stochasticity

```
ints = ["[1, 1000]", "[1001, 2000]", "[1, 1000]"]
for user in users:
  for interval in ints:
    prompt = template.format(user=user,interval=interval)
    response = model.sample(prompt)
    responses.append(response)
```

Assume we ensured independence across the outer loop iterations; a new question arises: within a single iteration of the outer loop, should the responses across rounds be independent for the same interval, e.g. for the repeated interval `[1,1000]`? Let the game dictate that across the rounds, users must always guess the same number if the interval remains unchanged. In other words, sampling within each iteration should be *repeatable*. To achieve this, an ad-hoc caching mechanism could be implemented, such as maintaining a dictionary to store the samples corresponding to each element in `ints`, reusing them when they appear a second time. However, this approach compromises readability and modularity, as it tightly couples the program logic to the cache implementation. Additionally, if different rounds require different numbers of samples, the complexity would increase further, potentially introducing bugs. Thus, to ensure consistent use of cached samples, one must specify scopes of the algorithm in which samples are expected to repeat, or be independent.

2.1 Cache Design Pattern

At Mnimi's core is the `Model` interface providing a single method

```
def sample(self, prompt: str) -> Iterator[str]
```

which takes in a prompt and returns an infinite sequence of i.i.d. responses from an LLM. The independence of samples within the sequence must be guaranteed by all implementations of this interface. In particular, we introduce three implementations of `Model` shown in Figure 2. Firstly, `ModelService` returns an iterator that queries the model provider for new samples, without caching. In this context, sample independence must be ensured by the provider.

Secondly, to realize a caching mechanism, we introduce a decorator `Repeatable`. It wraps an existing model so that its `sample` method produces a specialized type of iterator, `GetOldSample`. This iterator retrieves responses from a cache associated with each prompt; in case of cache misses, samples are obtained from the inner model, and added to the cache. Importantly, `Repeatable` still ensures independence of responses in the lazy sequence from a single call of `sample`. Instead, the repeatability is guaranteed across different calls of `sample`. To achieve that, `Repeatable` returns a new iterator over the same sequence for each call of `sample` with the same prompt. Figure 1 illustrates the impact of repeatability (Line 5), where even though `n3` has consumed `"131"`, the iterator returned to `n4_list` still begins at `"131"`. While `Repeatable` serves as an implementation of `Model`, it also abstracts the storage layer through the `store` and `load` methods, which must be implemented by concrete cache backends in `ConcreteCacheImplementation`.

Finally, to achieve statistical independence, we introduce a decorator `Independent`. In contrast to `Repeatable`, it ensures that each call to `sample` produces an independent sequence of responses. To achieve that, the implementation of `sample` for `Independent` merely always returns the same iterator for the same prompt. Since the iterator is shared, it is consumptive — responses used by one call will not be reused by others, making the effect stateful. Importantly, `Independent` itself neither generates nor stores responses; rather, it only defines a strategy for consuming them from the inner model. The use of `Independent` in Figure 1 in Line 8 ensures that since `n5_list` has already used `"131"` and `"561"`, the iterator for `n6_list` skips them and resamples, yielding `"452"` and `"980"`.

2.2 Design Pattern Application Examples

Here, we extend the example from the beginning of this section to three cases, illustrating how our design pattern elegantly handles successive changes in sampling constraints. At first, we assume that we have already cached a sequence of independent responses for each prompt (showing prompt template parameters only):

```
template("Alice", "[1, 1000]"):   ["2", "68", "109", "12"]
template("Bob",   "[1, 1000]"):   ["297", "573"]
template("Alice", "[1001, 2000]"):["1393", "1002"]
template("Bob",   "[1001, 2000]"):["1740"]
```

First, assume the user must be given the same number in every round, even in different games:

```
model = Persistent(model, "/cache/path")
responses = []
for user in users:
  for interval in ints:
    prompt = template.format(user=user,interval=interval)
    response = next(model.sample(prompt))
    responses.append(response)
assert responses == ["2", "1393", "2", "297", "1740", "297
    ", "2", "1393", "2"]
```

where `Persistent` is an subclass of `Repeatable` that caches responses on-disk. Now, assume a user is given an independent number every time they request one:

```
model = Persistent(model, "/cache/path")
model = Independent(model)
responses = []
for user in users:
  for interval in ints:
    prompt = template.format(user=user,interval=interval)
    response = next(model.sample(prompt))
    responses.append(response)
assert responses == ["2", "1393", "68", "297", "1740", "
    573", "109", "1002", "12"]
```

While the two examples above show trivial caching semantics supported by previous systems, Mnimi provides more fine-grained control over sample independence. To show it, assume a user must be given the same number in every round for the same interval, but across different games, corresponding to different iterations of the outer loop, numbers given to the same user must be independent. It requires composing repeatable and independent layers:

```
model = Persistent(model, "/cache/path")
model = Independent(model)
responses = []
for user in users:
  rep_model = InMemory(model)
```

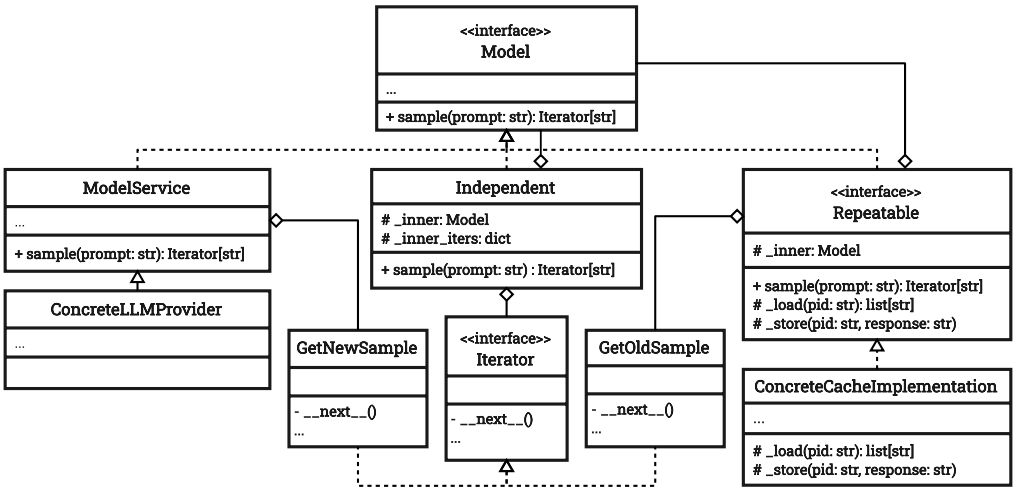


Figure 2: Structure of Mnimi design pattern that combines decorators and iterators over infinite sequences of responses to encode statistical independence constraints in a modular fashion. Mnimi provides two concrete implementations of `Repeatable`: an in-memory cache `InMemory` and an on-disk cache `Persistent`.

```
  for interval in ints:
    prompt = template.format(user=user,interval=interval)
    response = next(rep_model.sample(prompt))
    responses.append(response)
assert responses == ["2", "1393", "2", "297", "1740", "297
    ", "68", "1002", "68"]
```

where `InMemory`, a subclass of `Repeatable` with in-memory cache, ensures deterministic reuse within the loop, `Independent` ensures independent sampling across loop iterations, and `Persistent` provides on-disk cache for reproducibility across runs. Here, statistical independence constraints are elegantly encoded through LLM reference types. In contrast, systems like Cache Saver that rely on explicit namespaces prohibit intra-namespace reuse of samples, which prevents expressing repeatability within each loop iteration.

3 Case Study: Mnimi Integration in SpecFix

SpecFix [12] automatically repairs ambiguous programming problem descriptions to improve LLM-based code generation. It samples multiple candidate programs based on a given natural-language description, clusters their behaviors, and evaluates interpretive ambiguity using metrics such as semantic entropy and example consistency. The pipeline consists of several stages: test generation, program sampling, and iterative repair, all of which rely heavily on a large number of stochastic LLM calls. As a result, the process can produce varying outputs — including tests, programs, and final repaired descriptions — across different runs, making it expensive to reproduce and hard to debug. Meanwhile, the metrics driving SpecFix, such as semantic entropy and Pass@K, are fundamentally statistical and depend on independent samples. Furthermore, SpecFix's repair loop necessitates generation of independent repairs. These aspects make naive caching not applicable for SpecFix.

We applied Mnimi to SpecFix's source code, systematically encoding the required statistical independence constraints using LLM reference types. Each caching layer in SpecFix serves a distinct semantic purpose tailored to the statistical requirements of the repair process. Listing 3 presents a simplified version of SpecFix's repair loop, which iteratively refines a given description over a maximum of `max_attempts`. If the score for a repaired description fails to improve, the repair is discarded. This last aspect of the algorithm introduces specific requirements for statistical independence in LLM samples. While stable score computation for each description is desirable — achieved through the `Persistent` caching layer, which consistently provides the same sets of programs and tests for the same description — each repair attempt must remain independent. This independence is crucial to prevent the algorithm from becoming stuck in repeated unsuccessful attempts when the score fails to improve. Thus, we utilize an `Independent` model reference, which guarantees that each repair attempt explores fresh samples.

To quantify Mnimi's impact on *SpecFix*, we conducted eight controlled runs on 100 randomly selected HumanEval+ problems using `gpt-4.1-mini` at temperature 1.0. Each run varied the caching configuration and the program sample size (C) and test sample size (E), which estimate behavioral diversity, as summarized in Table 1. The first run, labeled **Baseline**, mirrors the original *SpecFix* behavior as no cached responses are used, with the small change of writing all outputs to the cache for future reuse. **No-cache** runs, in contrast, ignore caching entirely (no reads or writes), incurring full cost and runtime on every execution and forfeiting reproducibility.

Enabling reading from the baseline's cached responses immediately reduced query cost to zero on the next run, replaying results byte-for-byte. When the program sample size increased from C=20 to C=40, or from E=10 to E=20 the **First cached** run incurred additional cost only for the 20 new programs and the 10 extra tests respectively. The next **Cached replays** again dropped to zero cost, demonstrating perfect reuse once the cache was complete. Without caching, all generations are recomputed from scratch, greatly increasing both cost and runtime. This confirms Mnimi 's incremental cost model: identical workloads replay deterministically at zero cost, and only new generations trigger additional queries.

Table 1: Token usage, runtime, and cost of LLM queries in SpecFix with and without Mnimi caching (100 HumanEval+ tasks, `gpt-4.1-mini`, temperature 1.0). C is the number of generated programs, E is the number of generated tests. Baseline does not read from the cache but records all responses for future reuse, with its results reflecting how Specfix behaved originally. No cache runs ignore caching entirely (no reads or writes). First cached reuses stored results from the baseline, paying only for unseen generations, and Cached replay deterministically replays all queries at zero cost.

Run configuration	Prompt tokens	Completion tokens	Total tokens	API time (s)	Total cost ($)
Baseline (c=20, e=10)	97,669	319,814	417,483	1,716.1	1.10
Cached replay (c=20, e=10)	**0**	**0**	**0**	**0.0**	**0.00**
No cache (c=40, e=10)	100,733	500,792	601,525	2,344.9	1.68
First cached (c=40, e=10)	50,885	250,349	301,234	867.5	0.84
Cached replay (c=40, e=10)	**0**	**0**	**0**	**0.0**	**0.00**
No cache (c=20, e=20)	94,207	402,743	496,950	2,234.6	1.36
First cached (c=20, e=20)	2,941	32,872	35,813	93.9	0.11
Cached replay (c=20, e=20)	**0**	**0**	**0**	**0.0**	**0.00**

Figure 3: Statistical independence constraints in SpecFix repair loop encoded via Mnimi decorators.

```python
def specfix(problem, model, C=20, E=10, max_attempts=3):
    rep_model = Persistent(model, "/cache/path")
    ind_model = Independent(rep_model)

    def score(D):
        prompt = CODE_GEN_TMPL.format(description=D)
        # generate C programs, always the same
        # for the same requirements
        progs = islice(rep_model.sample(prompt), C)

        prompt = TEST_GEN_TMPL.format(description=D)
        # generate E tests, always the same
        # for the same requirements
        tests = islice(rep_model.sample(prompt), E)

        clusters = cluster(progs, tests)
        return evaluate_entropy_and_consistency(clusters)

    D = problem.description

    for attempt in range(max_attempts):
        prompt = REPAIR_TMPL.format(description=D)
        # repairs are always independent, but the sequence
        # of repairs is cached on disk, and replayed
        D2 = next(ind_model.sample(prompt))

        if score(D2) > score(D):
            D = D2
        if score(D) >= THRESHOLD:
            return D
    return D
```

Moreover, reproducible cached runs allowed all experiments to be re-run in seconds without recontacting the API.

Reproducibility is a critical challenge in empirical computer science, especially for tools like SpecFix that rely on stochastic processes such as sampling from LLMs. Without mechanisms like caching and deterministic replay, researchers must rerun entire experiments from scratch, making replication studies prohibitively expensive in time and cost, and exacerbating the replication crisis in the field [7]. Local caching addresses this issue by reducing computational overhead, enabling stable and deterministic outcomes, and facilitating data sharing for validation and comparison.

4 Discussion

While Mnimi enhances reproducibility and reduces costs at negligible overhead, several challenges remain open. Future extensions could integrate semantic caching [10], allowing re-use across paraphrased or structurally equivalent prompts while still preserving statistical guarantees. Introducing concurrency-aware cache access would further accelerate large-scale multi-threaded workflows, enabling safe parallelism across repair attempts and datasets.

The current implementation has limitations. Its correctness relies on deterministic serialization of prompts and settings; any nondeterminism, such as API-side randomness or floating-point ordering, can introduce subtle drift. Cache storage may grow large for tasks with high prompt diversity, and although disk-based persistence is efficient, long-term cache management and eviction remain manual.

5 Related Work

Traditional Caching. Caching is crucial for system optimization but is challenging to implement correctly. Besnard et al. [3] highlighted that caching non-deterministic functions can alter program behavior and proposed memoizing only pure, deterministic computations. This principle extends to web systems, where HTTP caching and CDN policies often exclude dynamic content. Bos et al. [4] introduced CacheCard, a NIC-level caching system that avoids caching dynamic web pages unless dependencies are explicitly tracked. In databases, Garrod et al. [9] described a distributed query result cache that ensures semantic correctness through invalidation triggered by database writes.

Probabilistic Sampling and Lazy Evaluation. Functional programming research has explored how to represent stochastic computations as reproducible, composable data structures. Scibior et al. [19] formalize probabilistic sampling as lazy sequences, enabling

repeatable access to random draws within purely functional programs. This serves as an inspiration for Mnimi that uses infinite lazy sequences to implement composable layers representing different statistical independence constraints.

Frameworks for LLM Programming and LLM Caching. Due to existence of LLM-related implementations, LLM-integrated scripts are facing challenges in terms of performance, modularity, and readability. Wang [21] proposed the use of composable effect handling to separate workflow logic from LLM-related effectful operations, thereby enabling modularity without sacrificing the potential for performance optimization. Consistent with this, our work also aims to benefit LLM programming by decoupling LLM caching from core program logic. Frameworks like LangChain [5] use string-based caching, where identical prompts return stored outputs to avoid redundant queries, improving efficiency but lacking control over statistical independence. Meanwhile, MeanCache [10] and GPTCache [2] extend caching to semantically similar prompts, complementing Mnimi. Frameworks like FloTorch [1] optimize agentic workflows by caching intermediate steps with per-step policies. Low-level optimizations, such as vLLM's key–value cache [14], accelerate token-level computation reuse but do not provide statistical independence control. Cache Saver [18] introduces "namespaces", where samples labelled with the same namespace are guaranteed to be independent. Section 2.2 shows examples that are hard to express using namespaces, but are concisely expressible in Mnimi.

6 Conclusion

This work introduces Mnimi, a cache design pattern that ensures statistical integrity while supporting modular LLM workflows. It encodes statistical independence constraints through reference types: repeatable model references yield identical sequences of responses for a given prompt, while independent references produce fresh, independent sequences of responses. By allowing flexible transformations between reference types, Mnimi simplifies maintaining statistical integrity across algorithm scopes. A case study on the SpecFix tool demonstrates how Mnimi reduces research costs while enhancing reproducibility and experimentation.

Acknowledgments

This work was sponsored by the National Natural Science Foundation of China (NSFC) under Grant No. W2542035.

References

[1] 2025. FloTorch: Agent-Aware Caching and Modular AI Pipelines. https://flotorch.ai. Accessed: 2025-10-13.

[2] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, Liling Tan, Dmitrijs Milajevs, Geeticka Chauhan, Jeremy Gwinnup, and Elijah Rippeth (Eds.). Association for Computational Linguistics, Singapore, 212–218. doi:10.18653/v1/2023.nlposs-1.24

[3] Loïc Besnard, Pedro Pinto, Imane Lasri, João Bispo, Erven Rohou, and João M.P. Cardoso. 2019. A framework for automatic and parameterizable memoization. *SoftwareX* 10 (2019), 100322. doi:10.1016/j.softx.2019.100322

[4] Herbert Bos and Kaiming Huang. 2009. CacheCard: caching static and dynamic content on the NIC. In *Proceedings of the 5th ACM/IEEE Symposium on Architectures for Networking and Communications Systems* (Princeton, New Jersey) *(ANCS '09)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/1882486.1882491

[5] Harrison Chase. 2023. LangChain: Building applications with large language models through composability. https://python.langchain.com. Accessed: 2025-10-13.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[7] Andy Cockburn, Pierre Dragicevic, Lonni Besançon, and Carl Gutwin. 2020. Threats of a replication crisis in empirical computer science. *Commun. ACM* 63, 8 (2020), 70–79.

[8] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1995. *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH.

[9] Charles Garrod, Amit Manjhi, Anastasia Ailamaki, Bruce Maggs, Todd Mowry, Christopher Olston, and Anthony Tomasic. 2008. Scalable query result caching for web applications. *Proc. VLDB Endow.* 1, 1 (Aug. 2008), 550–561. doi:10.14778/1453856.1453917

[10] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. 2025. MeanCache: User-Centric Semantic Caching for LLM Web Services. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 1298–1310. doi:10.1109/ipdps64566.2025.00117

[11] Christian Hobelsberger, Theresa Winner, Andreas Nawroth, Oliver Mitevski, and Anna-Carolina Haensch. 2025. Systematic Evaluation of Uncertainty Estimation Methods in Large Language Models. arXiv:2510.20460 [cs.CL] https://arxiv.org/abs/2510.20460

[12] Haoxiang Jia, Robbie Morris, He Ye, Federica Sarro, and Sergey Mechtaev. 2025. Automated Repair of Ambiguous Problem Descriptions for LLM-Based Code Generation. arXiv:2505.07270 [cs.SE] https://arxiv.org/abs/2505.07270

[13] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664* (2023).

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] https://arxiv.org/abs/2309.06180

[15] Qing Lyu, Kumar Shridhar, Chaitanya Malaviya, Li Zhang, Yanai Elazar, Niket Tandon, Marianna Apidianaki, Mrinmaya Sachan, and Chris Callison-Burch. 2025. Calibrating large language models with sample consistency. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 19260–19268.

[16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 [cs.CL] https://arxiv.org/abs/2303.17651

[17] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. arXiv:2303.08896 [cs.CL] https://arxiv.org/abs/2303.08896

[18] Nearchos Potamitis, Lars Henning Klein, Chongyang Xu, Attreyee Mukherjee, Bardia Mohammadi, Niket Tandon, Laurent Bindschaedler, and Akhil Arora. [n. d.]. Cache Saver: A Modular Framework for Efficient, Affordable, and Reproducible LLM Inference. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*.

[19] Adam Ścibior, Zoubin Ghahramani, and Andrew D Gordon. 2015. Practical probabilistic programming with monads. In *Proceedings of the 2015 ACM SIGPLAN Symposium on Haskell*. 165–176.

[20] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI] https://arxiv.org/abs/2303.11366

[21] Di Wang. 2025. Composable Effect Handling for Programming LLM-Integrated Scripts. In *Language Models and Programming Languages (LMPL'25)*. doi:10.1145/3759425.3763396